

CURSO 1: FUNDAMENTOS BÁSICOS DE PYTHON

CLASE 07 • NIVEL 1 - PRINCIPIANTE

Clase 07: Funciones, Parámetros y Scope

«Funciones como Máquinas Reutilizables de una Fábrica»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 07** del **Curso 1: Fundamentos Básicos de Python**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

🎯 Objetivos de la Sesión

Competencia Conceptual: Comprender la modularización, paso de parámetros por asignación, valores de retorno y ámbito (LEGB).

Competencia Práctica: Escribir funciones puras, documentadas con docstrings y parámetros opcionales.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 07: Funciones, Parámetros y Scope

Las funciones son bloques de código reutilizables diseñados para realizar una tarea específica.

☀ Metáfora Central: Funciones como Máquinas Reutilizables de una Fábrica

Una función es como un electrodoméstico: introduces ingredientes (argumentos) y recibes el resultado (return).

Principios Teóricos y Modelo Mental

Principio DRY (Don't Repeat Yourself): Si repites código, conviértelo en una función.

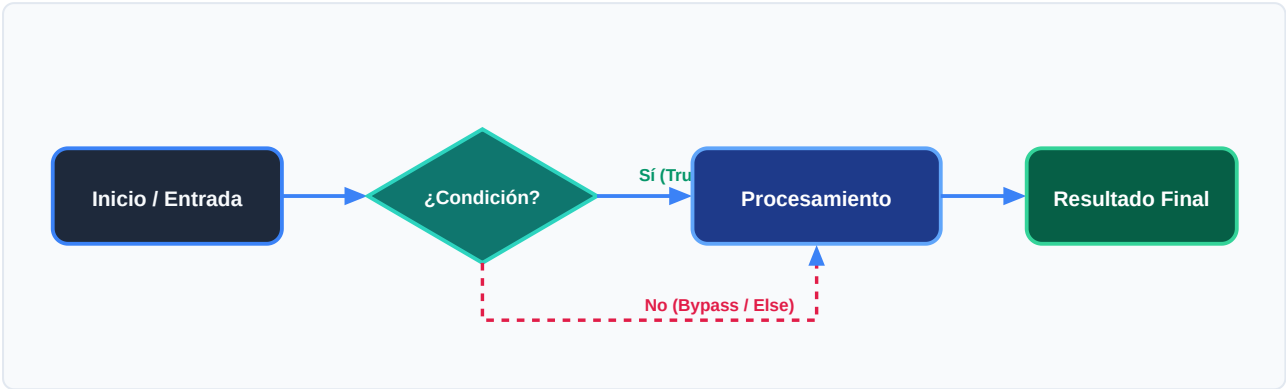
Regla de Scope LEGB: Python busca variables en orden: Local -> Enclosing -> Global -> Built-in.

⚡ Regla de Oro en Python

Toda función debe tener una sola responsabilidad clara.

Arquitectura y Movimiento de Datos en Memoria

Pila de llamadas (Call Stack), paso de argumentos y retorno de valores.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Definición y carga del objeto función.	Code object en memoria.
2. Evaluación	Invocación y creación del Stack Frame.	Variables locales enlazadas.
3. Transformación	Ejecución del cuerpo de la función.	Expresiones evaluadas.
4. Retorno / Salida	Sentencia return y destrucción del frame.	Retorno entregado.

Visualización Mental

Las variables locales creadas dentro de una función desaparecen cuando la función termina.

Código de Demostración en Python 3.10+

Función tipada con documentación y parámetros por defecto:

main.py (Python 3.10+)

PEP 8 Compliant

```
def calcular_precio_final(base: float, descuento_pct: float = 0.0, iva_pct: float = 21.0) -> float:
    """Calcula el importe total tras aplicar descuento e impuestos."""
    subtotal = base * (1 - descuento_pct / 100)
    total = subtotal * (1 + iva_pct / 100)
    return round(total, 2)

print("Total:", calcular_precio_final(100.0, descuento_pct=10.0))
```

Análisis de Ingeniería del Código

Uso de Type hints, valores por defecto en parámetros y docstring descriptivo.

Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Uno de los bugs más famosos de Python: el argumento por defecto mutable.

⚠ Gotcha Frecuente (Trampa de Principiante)

Usar listas o diccionarios vacíos como valores por defecto en la firma.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
def agregar(item, lista=[]): # ❌ Se comparte
    entre llamadas
    lista.append(item)
    return lista
```

✅ Patrón Correcto

```
def agregar(item, lista=None): # ✅ Inmutable
    None
    if lista is None: lista = []
    lista.append(item)
    return lista
```

🛡 Consejo de Resiliencia en Producción

Mantén las funciones puras evitando modificar variables globales directamente.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 07: Funciones, Parámetros y Scope**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Funciones como Máquinas Reutilizables de una Fábrica
2. Regla de Oro	Toda función debe tener una sola responsabilidad clara.
3. Gotcha Crítico	Usar listas o diccionarios vacíos como valores por defecto en la firma.
4. Buenas Prácticas	Mantén las funciones puras evitando modificar variables globales directamente.

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Escribe una función que reciba una lista de números y retorne el mínimo, el máximo y el promedio.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.